STAR RX72N - uart_test Bench Test

1.0.0

Generated by Doxygen 1.16.1

Chapter 1

Directory Hierarchy

1.1 Directories

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

Chapter 3

Directory Documentation

3.1 uart_test Directory Reference

Files

- file clock.c

  RX72N clock init for the STAR PCB bench test: MOSC 24 MHz crystal -> PLL 240 MHz -> ICLK=120 / PCKA=120 / PCKB=60 / FCLK=60 MHz.
- file linker.ld
- file main.c

  SCI9 UART verification – prints ”TICK 0xNNNN\n” every second.
- file sci9.c

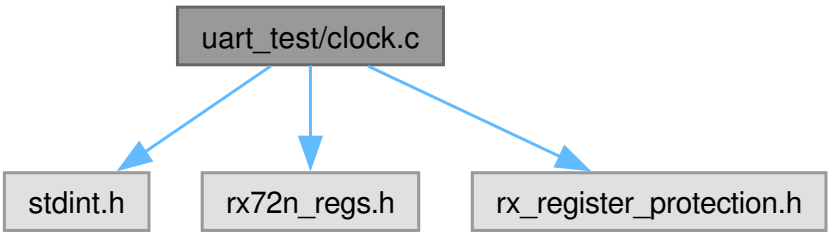
Chapter 4

File Documentation

4.1 uart_test/clock.c File Reference

RX72N clock init for the STAR PCB bench test: MOSC 24 MHz crystal -> PLL 240 MHz -> ICLK=120 / PCKA=120 / PCKB=60 / FCLK=60 MHz.

#include <stdint.h>
#include "rx72n_regs.h"
#include "rx_register_protection.h"
Include dependency graph for clock.c:



Enumerations

- enum clock_extra_addrs_t : uintptr_t { k_packcr_addr = 0x00080044U }
- enum clock_word_constants_t : uint16_t { k_pllcr_mosc_x10 = 0x1300U }
- enum clock_sckcr_t : uint32_t { k_sckcr_value = 0x21C21211U }
- enum clock_byte_constants_t : uint8_t {
  k_sub_clock_stopped = 0x01U , k_main_osc_enabled = 0x00U , k_pll_enabled = 0x00U , k_oscovfsr_moovf = 0x01U ,
  k_oscovfsr_plovf = 0x04U }
- enum clock_timeout_t : uint32_t { k_main_osc_poll_max = 500000U , k_pll_lock_poll_max = 500000U }

Functions

- void clock_init (void)

4.1.1 Detailed Description

RX72N clock init for the STAR PCB bench test: MOSC 24 MHz crystal -> PLL 240 MHz -> ICLK=120 / PCKA=120 / PCKB=60 / FCLK=60 MHz.

Self-contained reduced version of src/rx_clock_power_init.c, stripped of the logging, asserts, simulator branches, and PPLL/USB clock paths the uart test does not need. Configures only the main PLL.

Path: EXTAL 24 MHz -> MOSC -> PLL (x10 / 1) = 240 MHz -> SCKCR=0x21C21211. After this routine returns, ICLK is 120 MHz (half of production's 240 MHz – the bench test runs the CPU at reduced speed to cut power dissipation) and PCLKA (the SCI/GPTW source for this test) is 120 MHz, matching k_pclka_hz in libs/rx_hal/inc/rx72n_clock.h.

SPDX-License-Identifier: MIT

Copyright

Definition in file clock.c.

4.1.2 Enumeration Type Documentation

4.1.2.1 clock_byte_constants_t

enum clock_byte_constants_t : uint8_t

Enumerator

k_sub_clock_stopped	SOSCCR: stop sub-clock oscillator
k_main_osc_enabled	MOSCCR: 0 = MOSC running
k_pll_enabled	PLLCR2: 0 = PLL running
k_oscovfsr_moovf	OSCOVFSR bit 0: MOSC stable
k_oscovfsr_plovf	OSCOVFSR bit 2: PLL stable

Definition at line 60 of file clock.c.

```
00060             : uint8_t {
00061   k_sub_clock_stopped = 0x01U, /**< SOSCCR: stop sub-clock oscillator */
00062   k_main_osc_enabled  = 0x00U, /**< MOSCCR: 0 = MOSC running */
00063   k_pll_enabled       = 0x00U, /**< PLLCR2: 0 = PLL running */
00064   k_oscovfsr_moovf    = 0x01U, /**< OSCOVFSR bit 0: MOSC stable */
00065   k_oscovfsr_plovf    = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00066 } clock_byte_constants_t;
```

4.1.2.2 clock_extra_addrs_t

enum clock_extra_addrs_t : uintptr_t

Enumerator

k_packcr_addr	Peripheral clock source: 0=PLL

Definition at line 29 of file clock.c.

```
00029             : uintptr_t {
00030   k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL */
00031 } clock_extra_addrs_t;
```

4.1.2.3 clock_sckcr_t

enum clock_sckcr_t : uint32_t

Enumerator

k_sckcr_value	

Definition at line 51 of file clock.c.

```
00051             : uint32_t {
00052   /*
00053    * SCKCR dividers (bench-test config, ICLK half-speed vs production):
00054    *   FCK=/4 (60), ICK=/2 (120), PSTOP0=1, PSTOP1=1, BCK=/4 (60),
00055    *   PCKA=/2 (120), PCKB=/4 (60), PCKC=/2 (120), PCKD=/2 (120).
00056    */
00057   k_sckcr_value = 0x21C21211U,
00058 } clock_sckcr_t;
```

4.1.2.4 clock_timeout_t

enum clock_timeout_t : uint32_t

Enumerator

k_main_osc_poll_max	
k_pll_lock_poll_max	

Definition at line 68 of file clock.c.

```
00068             : uint32_t {
00069   /* Bounded poll counts (NASA Rule 2). We poll the hardware ready flag
00070    * (OSCOVFSR.MOOVF / .PLOVF) so the actual wait is hardware-determined --
00071    * roughly 10 ms physical time regardless of CPU clock. The upper bound
00072    * just prevents an infinite spin if the crystal is missing. ~500k iters
00073    * at LOCO 240 kHz is ~3 s, which is still much shorter than the prior
00074    * fixed 10-second busy-wait. */
00075   k_main_osc_poll_max  = 500000U,
00076   k_pll_lock_poll_max  = 500000U,
00077 } clock_timeout_t;
```

4.1.2.5 clock_word_constants_t

enum clock_word_constants_t : uint16_t

Enumerator

k_pllcr_mosc_x10	

Definition at line 36 of file clock.c.

```
00036             : uint16_t {
00037   /*
00038    * PLLCR layout:
00039    *   bits[1:0]  PLIDIV    : 00 = /1
00040    *   bit [4]    PLLSRCSEL : 0 = MOSC, 1 = HOCO
00041    *   bits[13:8] STC       : multiplier = (STC + 1) / 2
00042    *
00043    * MOSC 24 MHz / 1 * 10 = 240 MHz. STC = (10 * 2) - 1 = 19 = 0x13.
00044    * PLLSRCSEL = 0 (MOSC) -> bit 4 clear.
00045    *
00046    * Production rx_clock_power_init.c uses k_pll_multiplier_10_div_1 = 0x1300.
00047    */
00048   k_pllcr_mosc_x10 = 0x1300U,
00049 } clock_word_constants_t;
```

4.1.3 Function Documentation

4.1.3.1 clock_init()

void clock_init (
                void)

Definition at line 82 of file clock.c.

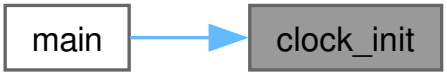
```
00083 {
00084   volatile rx_system_regs_t *sys = system_regs();
00085
00086   /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00087   *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00088
00089   /* Stop sub-clock oscillator -- not used on STAR. */
00090   sys->sosccr = k_sub_clock_stopped;
00091
00092   /* Start MOSC (24 MHz external crystal). */
00093   sys->mosccr = k_main_osc_enabled;
00094
00095   /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00096    * settled (typically ~10 ms physical time) instead of waiting a fixed
00097    * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00098   for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00099     if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
00100       break;
00101     }
00102   }
00103
00104   /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00105   sys->pllcr  = k_pllcr_mosc_x10;
00106   sys->pllcr2 = k_pll_enabled;
00107
00108   /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00109    * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
00110    * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00111    * can see we got past clock_init. */
00112   bool pll_locked = false;
00113   for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
00114     if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
00115       pll_locked = true;
00116       break;
00117     }
00118   }
00119
00120   if (pll_locked) {
00121     /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00122     *memwait_reg() = k_memwait_one_wait;
00123
```

```
00124     /* Apply system clock dividers. */
00125     sys->sckcr = k_sckcr_value;
00126
00127     /* Route UCLK from main PLL (UPLLSEL=0). */
00128     *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00129
00130     /* Switch system clock source to PLL. */
00131     sys->sckcr3 = k_sckcr3_cksel_pll;
00132   }
00133
00134   /* Re-lock PRCR. */
00135   *prcr_reg() = k_rx_prcr_lock;
00136 }
```

References k_main_osc_enabled, k_main_osc_poll_max, k_oscovfsr_moovf, k_oscovfsr_plovf, k_packcr_addr, k_pll_enabled, k_pll_lock_poll_max, k_pllcr_mosc_x10, k_sckcr_value, and k_sub_clock_stopped.

Referenced by main().

Here is the caller graph for this function:



4.2 clock.c

Go to the documentation of this file.

```
00001 /**
00002  * @file clock.c
00003  * @brief RX72N clock init for the STAR PCB bench test: MOSC 24 MHz crystal
00004  *        -> PLL 240 MHz -> ICLK=120 / PCKA=120 / PCKB=60 / FCLK=60 MHz.
00005  *
00006  * @details
00007  * Self-contained reduced version of src/rx_clock_power_init.c, stripped of
00008  * the logging, asserts, simulator branches, and PPLL/USB clock paths the
00009  * uart test does not need. Configures only the main PLL.
00010  *
00011  * Path: EXTAL 24 MHz -> MOSC -> PLL (x10 / 1) = 240 MHz -> SCKCR=0x21C21211.
00012  * After this routine returns, ICLK is 120 MHz (half of production's 240 MHz
00013  * -- the bench test runs the CPU at reduced speed to cut power dissipation)
00014  * and PCLKA (the SCI/GPTW source for this test) is 120 MHz, matching
00015  * k_pclka_hz in libs/rx_hal/inc/rx72n_clock.h.
00016  *
00017  * SPDX-License-Identifier: MIT
00018  * @copyright Copyright (c) 2026 Locked Inc.
00019  */
00020
00021 #include <stdint.h>
00022
00023 #include "rx72n_regs.h"
00024 #include "rx_register_protection.h"
00025
00026 /*
      ============================================================================
00027  * Register addresses not exposed by rx_system_regs_t
00028  *
      ============================================================================
      */
00029 typedef enum : uintptr_t {
00030   k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL */
00031 } clock_extra_addrs_t;
00032
00033 /*
      ============================================================================
00034  * Bit / value constants (mirrors src/rx_clock_power_init.c)
00035  *
      ============================================================================
      */
00036 typedef enum : uint16_t {
00037   /*
00038    * PLLCR layout:
00039    *   bits[1:0]  PLIDIV    : 00 = /1
```

```
00040  *    bit [4]    PLLSRCSEL : 0 = MOSC, 1 = HOCO
00041  *    bits[13:8] STC       : multiplier = (STC + 1) / 2
00042  *
00043  * MOSC 24 MHz / 1 * 10 = 240 MHz. STC = (10 * 2) - 1 = 19 = 0x13.
00044  * PLLSRCSEL = 0 (MOSC) -> bit 4 clear.
00045  *
00046  * Production rx_clock_power_init.c uses k_pll_multiplier_10_div_1 = 0x1300.
00047  */
00048  k_pllcr_mosc_x10 = 0x1300U,
00049 } clock_word_constants_t;
00050
00051 typedef enum : uint32_t {
00052  /*
00053  * SCKCR dividers (bench-test config, ICLK half-speed vs production):
00054  *   FCK=/4 (60), ICK=/2 (120), PSTOP0=1, PSTOP1=1, BCK=/4 (60),
00055  *   PCKA=/2 (120), PCKB=/4 (60), PCKC=/2 (120), PCKD=/2 (120).
00056  */
00057  k_sckcr_value = 0x21C21211U,
00058 } clock_sckcr_t;
00059
00060 typedef enum : uint8_t {
00061  k_sub_clock_stopped = 0x01U, /**< SOSCCR: stop sub-clock oscillator */
00062  k_main_osc_enabled  = 0x00U, /**< MOSCCR: 0 = MOSC running */
00063  k_pll_enabled       = 0x00U, /**< PLLCR2: 0 = PLL running */
00064  k_oscovfsr_moovf    = 0x01U, /**< OSCOVFSR bit 0: MOSC stable */
00065  k_oscovfsr_plovf    = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00066 } clock_byte_constants_t;
00067
00068 typedef enum : uint32_t {
00069  /* Bounded poll counts (NASA Rule 2). We poll the hardware ready flag
00070  * (OSCOVFSR.MOOVF / .PLOVF) so the actual wait is hardware-determined --
00071  * roughly 10 ms physical time regardless of CPU clock. The upper bound
00072  * just prevents an infinite spin if the crystal is missing. ~500k iters
00073  * at LOCO 240 kHz is ~3 s, which is still much shorter than the prior
00074  * fixed 10-second busy-wait. */
00075  k_main_osc_poll_max  = 500000U,
00076  k_pll_lock_poll_max  = 500000U,
00077 } clock_timeout_t;
00078
00079 /*
      ============================================================================
00080 * clock_init -- bring the chip from POR defaults to PLL 240 / PCKA 120 MHz.
00081 *
      ============================================================================
      */
00082 void clock_init(void)
00083 {
00084  volatile rx_system_regs_t *sys = system_regs();
00085
00086  /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00087  *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00088
00089  /* Stop sub-clock oscillator -- not used on STAR. */
00090  sys->sosccr = k_sub_clock_stopped;
00091
00092  /* Start MOSC (24 MHz external crystal). */
00093  sys->mosccr = k_main_osc_enabled;
00094
00095  /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00096  * settled (typically ~10 ms physical time) instead of waiting a fixed
00097  * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00098  for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00099   if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
00100    break;
00101   }
00102  }
00103
00104  /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00105  sys->pllcr  = k_pllcr_mosc_x10;
00106  sys->pllcr2 = k_pll_enabled;
00107
00108  /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00109  * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
00110  * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00111  * can see we got past clock_init. */
00112  bool pll_locked = false;
00113  for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
00114   if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
00115    pll_locked = true;
00116    break;
00117   }
00118  }
00119
00120  if (pll_locked) {
00121   /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00122   *memwait_reg() = k_memwait_one_wait;
00123
```

```
00124     /* Apply system clock dividers. */
00125     sys->sckcr = k_sckcr_value;
00126
00127     /* Route UCLK from main PLL (UPLLSEL=0). */
00128     *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00129
00130     /* Switch system clock source to PLL. */
00131     sys->sckcr3 = k_sckcr3_cksel_pll;
00132   }
00133
00134   /* Re-lock PRCR. */
00135   *prcr_reg() = k_rx_prcr_lock;
00136 }
```

4.3 uart_test/linker.ld File Reference

4.4 linker.ld

Go to the documentation of this file.

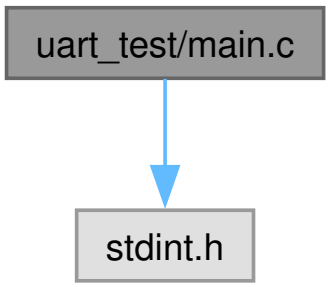
```
00001 /*
00002  * gpio_test/linker.ld -- Linker script for GPIO breakout board test.
00003  *
00004  * Same memory map as usb_test: 512 KB internal SRAM, 2 MB internal ROM.
00005  * Includes Renesas-syntax sections (P, B, D, C) needed by ThreadX RXv3 port.
00006  */
00007
00008 MEMORY
00009 {
00010     RAM : ORIGIN = 0x00000004, LENGTH = 0x7FFFC   /* 512 KB - 4 bytes */
00011     ROM : ORIGIN = 0xFFE00000, LENGTH = 0x200000  /* 2 MB             */
00012 }
00013
00014 SECTIONS
00015 {
00016     /* Code -- explicit ROM base so section VMA is anchored correctly. */
00017     .text 0xFFE00000 : AT(0xFFE00000)
00018     {
00019         *(.text*)
00020         *(P)
00021         *(P_1)
00022         *(.rodata*)
00023         *(C)
00024         *(C_1)
00025         *(C_2)
00026         etext = .;
00027     } > ROM
00028
00029     /*
00030      * Relocatable vector table -- startup.S loads INTB with this address.
00031      * Must be 4-byte aligned; only vectors 27 (SWINT) and 28 (CMT0) are
00032      * used, but allocate 256 entries (1024 bytes) for safety.
00033      */
00034     .rvectors ALIGN(4) :
00035     {
00036         _rvectors_start = .;
00037         KEEP(*(.rvectors))
00038         _rvectors_end = .;
00039     } > ROM
00040
00041     /*
00042      * Initialised data -- LMA stays in ROM, VMA in RAM.
00043      * startup.S copies _data..edata from _mdata on boot.
00044      */
00045     _mdata = .;
00046     .data : AT(_mdata)
00047     {
00048         _data = .;
00049         *(.data*)
00050         *(D)
00051         *(D_1)
00052         *(D_2)
00053         _edata = .;
00054     } > RAM
00055
00056     /* BSS -- zeroed by startup.S */
00057     .bss :
00058     {
00059         _bss = .;
```

```
00060         *(.bss*)
00061         *(COMMON)
00062         *(B)
00063         *(B_1)
00064         *(B_2)
00065         _ebss = .;
00066         . = ALIGN(4);
00067         _end = .;
00068     } > RAM AT>RAM
00069
00070     /*
00071      * Stacks -- USP grows down from top of RAM, ISP 4 KB below that.
00072      */
00073     _ustack = ORIGIN(RAM) + LENGTH(RAM);
00074     _istack = _ustack - 0x1000;
00075
00076     /*
00077      * Exception vector table at fixed hardware address 0xFFFFFF80.
00078      * 32 entries -- all unused.
00079      */
00080     .exvectors 0xFFFFFF80 : AT(0xFFFFFF80)
00081     {
00082         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00083         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00084         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00085         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00086         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00087         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00088         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00089         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00090     } > ROM
00091
00092     /* Fixed reset vector at 0xFFFFFFFC */
00093     .fvectors 0xFFFFFFFC : AT(0xFFFFFFFC)
00094     {
00095         KEEP(*(.fvectors))
00096     } > ROM
00097 }
```

4.5 uart_test/main.c File Reference

SCI9 UART verification – prints ”TICK 0xNNNN\n” every second.

#include <stdint.h>
Include dependency graph for main.c:



Enumerations

- enum timing_t : uint32_t { k_iters_per_ms = 240000U , k_tick_period_ms = 200U }

Functions

- void clock_init (void)
- void sci9_debug_init (void)
- void sci9_debug_puts (const char *s)
- void sci9_debug_puthex16 (uint16_t v)
- static void busy_wait_ms (uint32_t ms)
- int main (void)

4.5.1 Detailed Description

SCI9 UART verification – prints "TICK 0xNNNN\n" every second.

Boots, initializes the clock and SCI9 (TXD9 = PB7), then loops forever printing a hex tick counter with a 1-second cadence so the host can confirm a continuous stream on /dev/ttyACM0 @ 115200 8N1.

SPDX-License-Identifier: MIT

Definition in file main.c.

4.5.2 Enumeration Type Documentation

4.5.2.1 timing_t

enum timing_t : uint32_t

Enumerator

k_iters_per_ms	
k_tick_period_ms	

Definition at line 21 of file main.c.

```
00021             : uint32_t {
00022   k_iters_per_ms = 240000U,
00023   k_tick_period_ms = 200U, /* 5 ticks/sec so a short capture catches several */
00024 } timing_t;
```

4.5.3 Function Documentation

4.5.3.1 busy_wait_ms()

void busy_wait_ms (
uint32_t ms) [static]

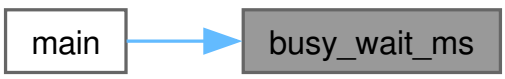
Definition at line 26 of file main.c.

```
00027 {
00028   for (uint32_t m = 0; m < ms; m++) {
00029     for (volatile uint32_t i = 0; i < k_iters_per_ms; i++) {
00030       __asm__ volatile("nop");
00031     }
00032   }
00033 }
```

References k_iters_per_ms.

Referenced by main().

Here is the caller graph for this function:



4.5.3.2 clock_init()

void clock_init (
void) [extern]

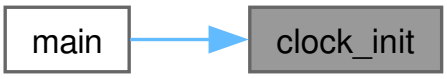
Definition at line 82 of file clock.c.

```
00083 {
00084   volatile rx_system_regs_t *sys = system_regs();
00085
00086   /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00087   *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00088
00089   /* Stop sub-clock oscillator -- not used on STAR. */
00090   sys->sosccr = k_sub_clock_stopped;
00091
00092   /* Start MOSC (24 MHz external crystal). */
00093   sys->mosccr = k_main_osc_enabled;
00094
00095   /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00096    * settled (typically ~10 ms physical time) instead of waiting a fixed
00097    * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00098   for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00099     if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
00100       break;
00101     }
00102   }
00103
00104   /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00105   sys->pllcr  = k_pllcr_mosc_x10;
00106   sys->pllcr2 = k_pll_enabled;
00107
00108   /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00109    * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
00110    * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00111    * can see we got past clock_init. */
00112   bool pll_locked = false;
00113   for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
00114     if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
00115       pll_locked = true;
00116       break;
00117     }
00118   }
00119
00120   if (pll_locked) {
00121     /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00122     *memwait_reg() = k_memwait_one_wait;
00123
00124     /* Apply system clock dividers. */
00125     sys->sckcr = k_sckcr_value;
00126
00127     /* Route UCLK from main PLL (UPLLSEL=0). */
00128     *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00129
00130     /* Switch system clock source to PLL. */
00131     sys->sckcr3 = k_sckcr3_cksel_pll;
00132   }
00133
00134   /* Re-lock PRCR. */
00135   *prcr_reg() = k_rx_prcr_lock;
00136 }
```

References k_main_osc_enabled, k_main_osc_poll_max, k_oscovfsr_moovf, k_oscovfsr_plovf, k_packcr_addr, k_pll_enabled, k_pll_lock_poll_max, k_pllcr_mosc_x10, k_sckcr_value, and k_sub_clock_stopped.

Referenced by main().

Here is the caller graph for this function:



4.5.3.3 main()

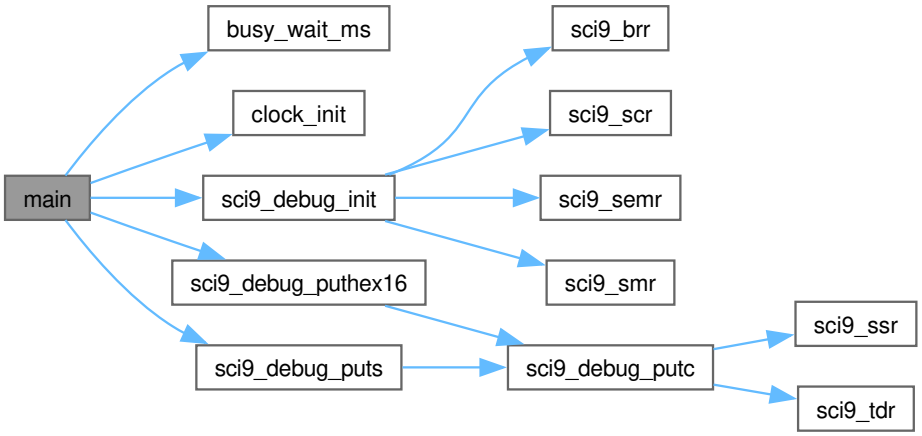
```
int main (
                void )
```

Definition at line 35 of file main.c.

```
00036 {
00037   clock_init();
00038   sci9_debug_init();
00039
00040   sci9_debug_puts("\nUART_TEST start (115200 8N1, TXD9=PB7)\n");
00041
00042   uint16_t count = 0;
00043   for (;;) {
00044     sci9_debug_puts("TICK ");
00045     sci9_debug_puthex16(count);
00046     sci9_debug_puts("\n");
00047     busy_wait_ms(k_tick_period_ms);
00048     count++;
00049   }
00050 }
```

References busy_wait_ms(), clock_init(), k_tick_period_ms, sci9_debug_init(), sci9_debug_puthex16(), and sci9_debug_puts().

Here is the call graph for this function:



4.5.3.4 sci9_debug_init()

```
void sci9_debug_init (
                void )   [extern]
```

Definition at line 104 of file sci9.c.

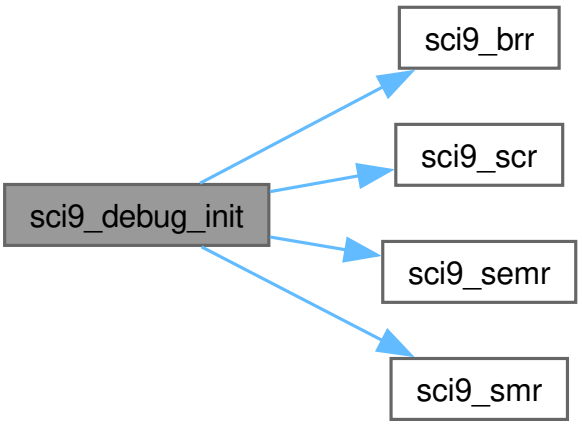
```
00105 {
00106     /* Step 1: release SCI9 from module stop. SCI9 lives at MSTPCRC bit 26
00107      * (NOT MSTPCRB.bit22 -- that's PDC). PRC1 unlock gates MSTPCR* writes. */
00108     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109     system_regs()->mstpcrc &= ~(uint32_t)(1UL << (uint32_t)k_mstpc_sci9_bit);
00110     *prcr_reg() = k_rx_prcr_lock;
00111
00112     /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC.  Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear B0WI */
00116     mpc()->pwpr   = k_pwpr_pfswe;     /* allow PFS writes */
00117     mpc()->pb6pfs = k_psel_sci9;
00118     mpc()->pb7pfs = k_psel_sci9;
00119     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear PFSWE */
00120     mpc()->pwpr   = k_pwpr_b0wi;      /* re-protect */
00121
```

```
00122     /* Step 3: PDR before PMR (production uart.c does this).  PB7 = output
00123      * for TX, PB6 = input for RX.  Then flip PMR to peripheral mode. */
00124     portb()->pdr |= (uint8_t)k_pb7_mask;
00125     portb()->pdr &= (uint8_t) ~k_pb6_mask;
00126     portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);
00127
00128     /* Step 4: SCI9 init.  Match the production order:
00129      *   SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130      * Don't touch SCMR (reset default 0xF2 is correct). */
00131     *sci9_scr() = k_scr_disabled;
00132     *sci9_smr() = k_smr_8n1_async;
00133     *sci9_brr() = k_brr_115200;
00134
00135     /* Wait roughly one bit time before enabling TX/RX. */
00136     for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
00137         __asm__ volatile("nop");
00138     }
00139
00140     *sci9_scr()  = k_scr_txrx_enabled;
00141     *sci9_semr() = k_semr_default;
00142 }
```

References k_brr_115200, k_brr_settle_loops, k_mstpc_sci9_bit, k_pb6_mask, k_pb7_mask, k_psel_sci9, k_pwpr_b0wi, k_pwpr_pfswe, k_pwpr_unlock_b0, k_scr_disabled, k_scr_txrx_enabled, k_semr_default, k_smr_8n1_async, sci9_brr(), sci9_scr(), sci9_semr(), and sci9_smr().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:

4.5.3.5 sci9_debug_puthex16()

```
void sci9_debug_puthex16 (
                uint16_t v)   [extern]
```

Definition at line 179 of file sci9.c.

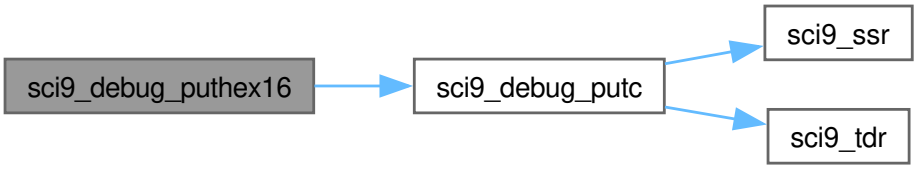
```
00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
```

```
00185         shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }
```

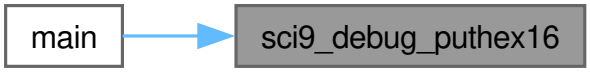
References k_hex_nibble_mask, k_hex_shift_init16, k_hex_shift_step, and sci9_debug_putc().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.3.6 sci9_debug_puts()

```
void sci9_debug_puts (
                const char * s)    [extern]
```

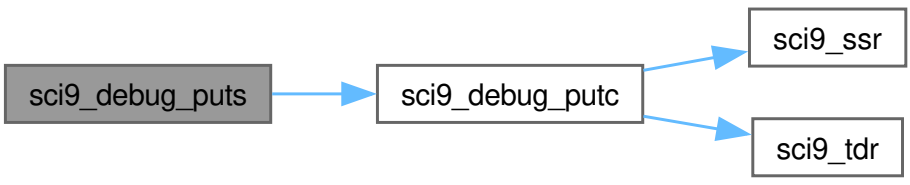
Definition at line 155 of file sci9.c.

```
00156 {
00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s++);
00165     }
00166 }
```

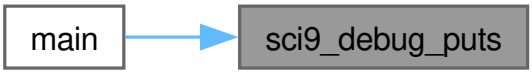
References sci9_debug_putc().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



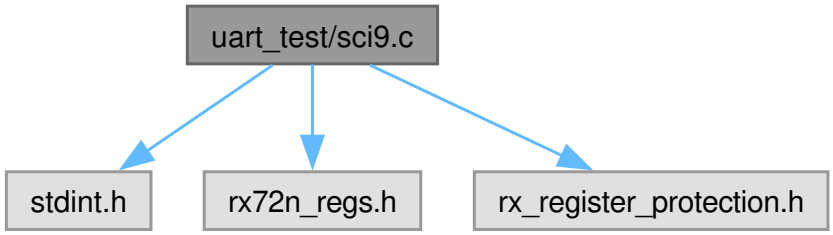
4.6 main.c

Go to the documentation of this file.

```
00001 /**
00002  * @file main.c
00003  * @brief SCI9 UART verification -- prints "TICK 0xNNNN\n" every second.
00004  *
00005  * @details
00006  * Boots, initializes the clock and SCI9 (TXD9 = PB7), then loops
00007  * forever printing a hex tick counter with a 1-second cadence so the
00008  * host can confirm a continuous stream on /dev/ttyACM0 @ 115200 8N1.
00009  *
00010  * SPDX-License-Identifier: MIT
00011  * @copyright Copyright (c) 2026 Locked Inc.
00012  */
00013
00014 #include <stdint.h>
00015
00016 extern void clock_init(void);
00017 extern void sci9_debug_init(void);
00018 extern void sci9_debug_puts(const char *s);
00019 extern void sci9_debug_puthex16(uint16_t v);
00020
00021 typedef enum : uint32_t {
00022   k_iters_per_ms = 240000U,
00023   k_tick_period_ms = 200U, /* 5 ticks/sec so a short capture catches several */
00024 } timing_t;
00025
00026 static void busy_wait_ms(uint32_t ms)
00027 {
00028   for (uint32_t m = 0; m < ms; m++) {
00029     for (volatile uint32_t i = 0; i < k_iters_per_ms; i++) {
00030       __asm__ volatile("nop");
00031     }
00032   }
00033 }
00034
00035 int main(void)
00036 {
00037   clock_init();
00038   sci9_debug_init();
00039
00040   sci9_debug_puts("\nUART_TEST start (115200 8N1, TXD9=PB7)\n");
00041
00042   uint16_t count = 0;
00043   for (;;) {
00044     sci9_debug_puts("TICK ");
00045     sci9_debug_puthex16(count);
00046     sci9_debug_puts("\n");
00047     busy_wait_ms(k_tick_period_ms);
00048     count++;
00049   }
00050 }
```

4.7 uart_test/sci9.c File Reference

#include <stdint.h>
#include "rx72n_regs.h"
#include "rx_register_protection.h"
Include dependency graph for sci9.c:



Enumerations

- enum sci9_addrs_t : uintptr_t { k_sci9_base = 0x000D0020U }
- enum sci9_cfg_t : uint8_t {
  k_psel_sci9 = 0x0AU , k_pb6_mask = (uint8_t)(1U << 6U) , k_pb7_mask = (uint8_t)(1U << 7U) , k_mstpc_sci9_bit = 26U ,
  k_pwpr_unlock_b0 = 0x00U , k_pwpr_pfswe = 0x40U , k_pwpr_b0wi = 0x80U , k_brr_115200 = 31U ,
  k_smr_8n1_async = 0x00U , k_scr_te_only = 0x20U , k_scr_txrx_enabled = 0x30U , k_scr_disabled = 0x00U ,
  k_ssr_tdre = 0x80U , k_scmr_default = 0xF2U , k_semr_default = 0x00U , k_hex_nibble_mask = 0x0FU }
- enum sci9_settle_t : uint16_t { k_brr_settle_loops = 1000U }
- enum hex_shifts_t : uint8_t { k_hex_shift_init32 = 28U , k_hex_shift_init16 = 12U , k_hex_shift_step = 4U }

Functions

- static volatile uint8_t * sci9_smr (void)
- static volatile uint8_t * sci9_brr (void)
- static volatile uint8_t * sci9_scr (void)
- static volatile uint8_t * sci9_tdr (void)
- static volatile uint8_t * sci9_ssr (void)
- static volatile uint8_t * sci9_scmr (void)
- static volatile uint8_t * sci9_semr (void)
- void sci9_debug_init (void)
- void sci9_debug_putc (char c)
- void sci9_debug_puts (const char *s)
- void sci9_debug_puthex32 (uint32_t v)
- void sci9_debug_puthex16 (uint16_t v)

4.7.1 Enumeration Type Documentation

4.7.1.1 hex_shifts_t

enum hex_shifts_t : uint8_t

Enumerator

k_hex_shift_init32	
k_hex_shift_init16	
k_hex_shift_step	

Definition at line 62 of file sci9.c.

```
00062             : uint8_t {
00063     k_hex_shift_init32 = 28U,
00064     k_hex_shift_init16 = 12U,
00065     k_hex_shift_step   = 4U,
00066 } hex_shifts_t;
```

4.7.1.2 sci9_addrs_t

enum sci9_addrs_t : uintptr_t

Enumerator

k_sci9_base	

Definition at line 35 of file sci9.c.

```
00035             : uintptr_t {
00036     k_sci9_base = 0x000D0020U,
00037 } sci9_addrs_t;
```

4.7.1.3 sci9_cfg_t

enum sci9_cfg_t : uint8_t

Enumerator

k_psel_sci9	SCI9 RXD/TXD function
k_pb6_mask	
k_pb7_mask	
k_mstpc_sci9_bit	
k_pwpr_unlock_b0	
k_pwpr_pfswe	
k_pwpr_b0wi	
k_brr_115200	
k_smr_8n1_async	
k_scr_te_only	CKE=00, MPIE=0, RIE=0, TE=1, RE=0
k_scr_txrx_enabled	CKE=00, MPIE=0, RIE=0, TE=1, RE=1
k_scr_disabled	
k_ssr_tdre	
k_scmr_default	
k_semr_default	
k_hex_nibble_mask	

Definition at line 39 of file sci9.c.

```
00039             : uint8_t {
00040     k_psel_sci9       = 0x0AU, /**< SCI9 RXD/TXD function */
00041     k_pb6_mask        = (uint8_t)(1U << 6U),
00042     k_pb7_mask        = (uint8_t)(1U << 7U),
00043     k_mstpc_sci9_bit  = 26U, /* MSTPCRC.MSTPC26 per RX72N HW manual page 410 */
00044     k_pwpr_unlock_b0  = 0x00U,
00045     k_pwpr_pfswe      = 0x40U,
00046     k_pwpr_b0wi       = 0x80U,
00047     k_brr_115200      = 31U, /* SCI9 uses PCLKA=120 MHz (SCI7-11 group); BRR=31 @ CKS=0 ABCS=0 -> ~117
        kbaud (1.7% over 115200) */
00048     k_smr_8n1_async   = 0x00U,
00049     k_scr_te_only     = 0x20U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=0 */
00050     k_scr_txrx_enabled = 0x30U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=1 */
00051     k_scr_disabled    = 0x00U,
00052     k_ssr_tdre        = 0x80U,
00053     k_scmr_default    = 0xF2U,
00054     k_semr_default    = 0x00U,
00055     k_hex_nibble_mask = 0x0FU,
00056 } sci9_cfg_t;
```

4.7.1.4 sci9_settle_t

enum sci9_settle_t : uint16_t

Enumerator

k_brr_settle_loops	Match production uart.c (~8.68 us @ 115200)

Definition at line 58 of file sci9.c.

```
00058             : uint16_t {
00059     k_brr_settle_loops = 1000U, /**< Match production uart.c (~8.68 us @ 115200) */
00060 } sci9_settle_t;
```

4.7.2 Function Documentation

4.7.2.1 sci9_brr()

volatile uint8_t * sci9_brr (
void) [inline], [static]

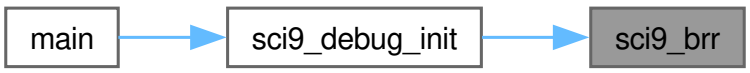
Definition at line 76 of file sci9.c.

```
00077 {
00078     return (volatile uint8_t *)(k_sci9_base + 1U);
00079 }
```

References k_sci9_base.

Referenced by sci9_debug_init().

Here is the caller graph for this function:



4.7.2.2 sci9_debug_init()

void sci9_debug_init (
void)

Definition at line 104 of file sci9.c.

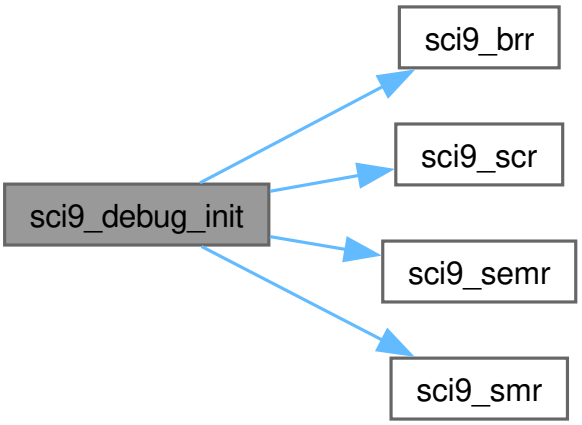
```
00105 {
00106     /* Step 1: release SCI9 from module stop. SCI9 lives at MSTPCRC bit 26
00107      * (NOT MSTPCRB.bit22 -- that's PDC). PRC1 unlock gates MSTPCR* writes. */
00108     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109     system_regs()->mstpcrc &= ~(uint32_t)(1UL << (uint32_t)k_mstpc_sci9_bit);
00110     *prcr_reg() = k_rx_prcr_lock;
00111
00112     /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC.  Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear B0WI */
00116     mpc()->pwpr   = k_pwpr_pfswe;     /* allow PFS writes */
00117     mpc()->pb6pfs = k_psel_sci9;
00118     mpc()->pb7pfs = k_psel_sci9;
00119     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear PFSWE */
00120     mpc()->pwpr   = k_pwpr_b0wi;      /* re-protect */
00121
```

```
00122     /* Step 3: PDR before PMR (production uart.c does this).  PB7 = output
00123      * for TX, PB6 = input for RX.  Then flip PMR to peripheral mode. */
00124     portb()->pdr |= (uint8_t)k_pb7_mask;
00125     portb()->pdr &= (uint8_t) ~k_pb6_mask;
00126     portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);
00127
00128     /* Step 4: SCI9 init.  Match the production order:
00129      *   SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130      * Don't touch SCMR (reset default 0xF2 is correct). */
00131     *sci9_scr() = k_scr_disabled;
00132     *sci9_smr() = k_smr_8n1_async;
00133     *sci9_brr() = k_brr_115200;
00134
00135     /* Wait roughly one bit time before enabling TX/RX. */
00136     for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
00137         __asm__ volatile("nop");
00138     }
00139
00140     *sci9_scr()  = k_scr_txrx_enabled;
00141     *sci9_semr() = k_semr_default;
00142 }
```

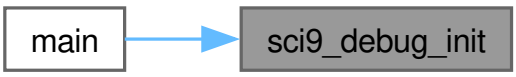
References k_brr_115200, k_brr_settle_loops, k_mstpc_sci9_bit, k_pb6_mask, k_pb7_mask, k_psel_sci9, k_pwpr_b0wi, k_pwpr_pfswe, k_pwpr_unlock_b0, k_scr_disabled, k_scr_txrx_enabled, k_semr_default, k_smr_8n1_async, sci9_brr(), sci9_scr(), sci9_semr(), and sci9_smr().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.3 sci9_debug_putc()

```
void sci9_debug_putc (
                char c)
```

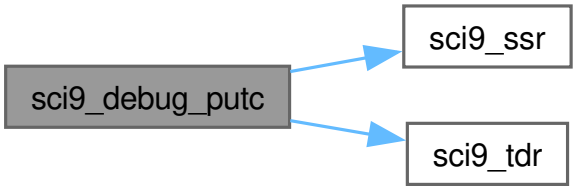
Definition at line 147 of file sci9.c.

```
00148 {
00149     while ((*sci9_ssr() & (uint8_t)k_ssr_tdre) == 0U) {
00150         /* spin */
00151     }
00152     *sci9_tdr() = (uint8_t)c;
00153 }
```

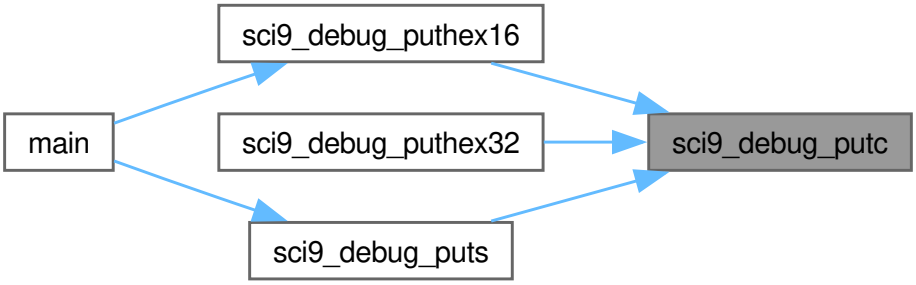
References k_ssr_tdre, sci9_ssr(), and sci9_tdr().

Referenced by sci9_debug_puthex16(), sci9_debug_puthex32(), and sci9_debug_puts().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.4 sci9_debug_puthex16()

```
void sci9_debug_puthex16 (
                uint16_t v)
```

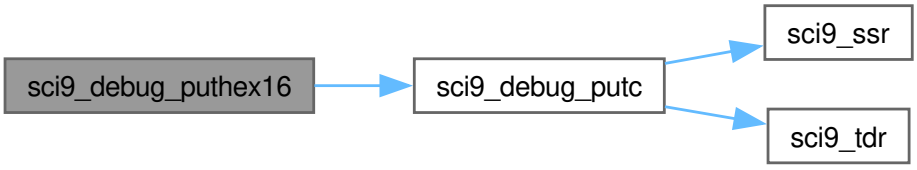
Definition at line 179 of file sci9.c.

```
00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
00185          shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v >> (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }
```

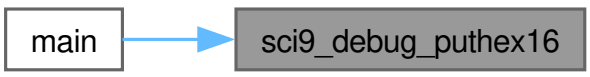
References k_hex_nibble_mask, k_hex_shift_init16, k_hex_shift_step, and sci9_debug_putc().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.5 sci9_debug_puthex32()

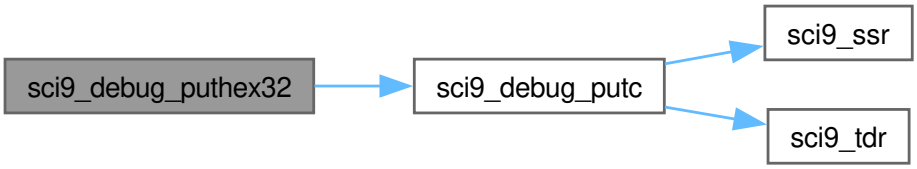
```
void sci9_debug_puthex32 (
                uint32_t v)
```

Definition at line 168 of file sci9.c.

```
00169 {
00170     static const char hex_digits[] = "0123456789ABCDEF";
00171     sci9_debug_putc('0');
00172     sci9_debug_putc('x');
00173     for (int8_t shift = (int8_t)k_hex_shift_init32; shift >= 0;
00174          shift -= (int8_t)k_hex_shift_step) {
00175         sci9_debug_putc(hex_digits[(v >> (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00176     }
00177 }
```

References k_hex_nibble_mask, k_hex_shift_init32, k_hex_shift_step, and sci9_debug_putc().

Here is the call graph for this function:



4.7.2.6 sci9_debug_puts()

```
void sci9_debug_puts (
                const char * s)
```

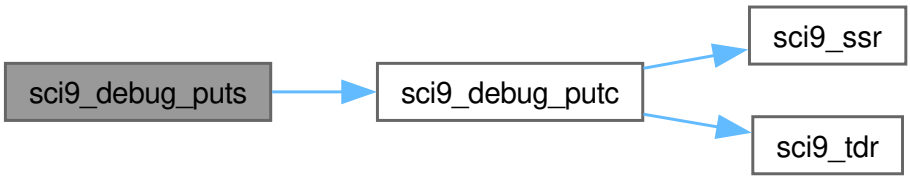
Definition at line 155 of file sci9.c.

```
00156 {
00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s++);
00165     }
00166 }
```

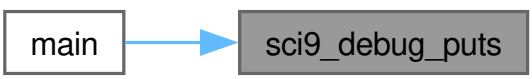
References sci9_debug_putc().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.7 sci9_scmr()

```
volatile uint8_t * sci9_scmr (
                void ) [inline], [static]
```

Definition at line 92 of file sci9.c.

```
00093 {
00094     return (volatile uint8_t *)(k_sci9_base + 6U);
00095 }
```

References k_sci9_base.

4.7.2.8 sci9_scr()

```
volatile uint8_t * sci9_scr (
                void ) [inline], [static]
```

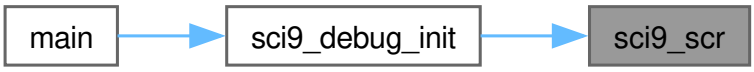
Definition at line 80 of file sci9.c.

```
00081 {
00082     return (volatile uint8_t *)(k_sci9_base + 2U);
00083 }
```

References k_sci9_base.

Referenced by sci9_debug_init().

Here is the caller graph for this function:



4.7.2.9 sci9_semr()

```
volatile uint8_t * sci9_semr (
                void ) [inline], [static]
```

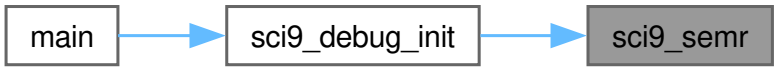
Definition at line 96 of file sci9.c.

```
00097 {
00098     return (volatile uint8_t *)(k_sci9_base + 7U);
00099 }
```

References k_sci9_base.

Referenced by sci9_debug_init().

Here is the caller graph for this function:



4.7.2.10 sci9_smr()

volatile uint8_t * sci9_smr (
void) [inline], [static]

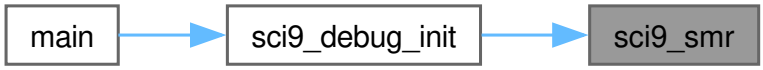
Definition at line 72 of file sci9.c.

```
00073 {
00074     return (volatile uint8_t *)(k_sci9_base + 0U);
00075 }
```

References k_sci9_base.

Referenced by sci9_debug_init().

Here is the caller graph for this function:



4.7.2.11 sci9_ssr()

volatile uint8_t * sci9_ssr (
void) [inline], [static]

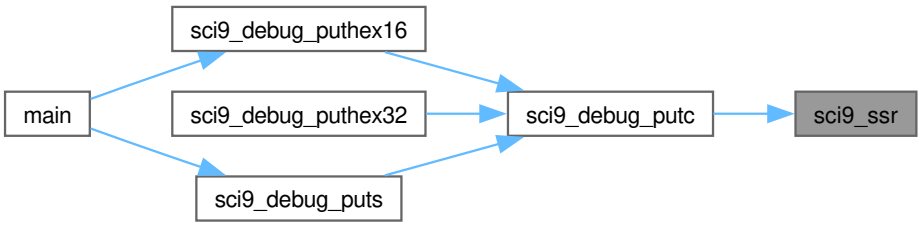
Definition at line 88 of file sci9.c.

```
00089 {
00090     return (volatile uint8_t *)(k_sci9_base + 4U);
00091 }
```

References k_sci9_base.

Referenced by sci9_debug_putc().

Here is the caller graph for this function:



4.7.2.12 sci9_tdr()

volatile uint8_t * sci9_tdr (
void) [inline], [static]

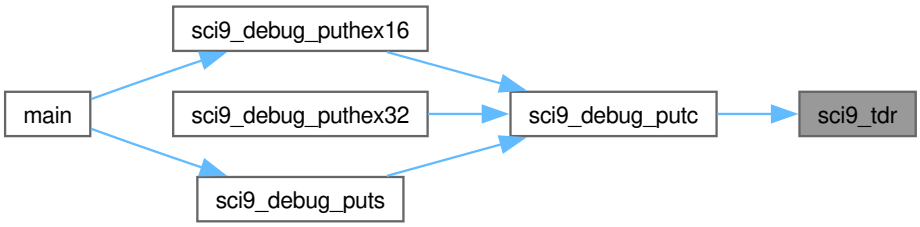
Definition at line 84 of file sci9.c.

```
00085 {
00086     return (volatile uint8_t *)(k_sci9_base + 3U);
00087 }
```

References k_sci9_base.

Referenced by sci9_debug_putc().

Here is the caller graph for this function:



4.8 sci9.c

Go to the documentation of this file.

```
00001 /**
00002  * @file sci9_debug.c
00003  * @brief Minimal polled-mode SCI9 (TXD9 = PB7, RXD9 = PB6) debug UART for uart_test.
00004  *
00005  * @details
00006  * The STAR board routes SCI9 to the on-board CY7C65213 USB-to-UART bridge
00007  * which appears as /dev/ttyACM0 on the host (or COMx on Windows).  This
00008  * file provides just enough init + putc/puts to emit ASCII status from
00009  * the firmware so we can debug the USB CDC bring-up without needing the
00010  * USB device to actually enumerate.
00011  *
00012  * Uses the rx_hal struct accessors (mpc(), portb(), system_regs(), prcr_reg())
00013  * so the addresses are all compiler-verified via static_assert.
00014  *
00015  * Baud = 115200. SCI9 is in the SCI7-11 group that uses PCLKA (120 MHz),
00016  * not PCLKB. At CKS=0, ABCS=0:
00017  *   BRR = (PCLKA / (32 * baud)) - 1 = 120000000 / 3686400 - 1 = 31.55 -> 31
00018  *   Actual baud = PCLKA / (32 * (BRR+1)) = 117187 Hz (+1.7% vs 115200, in tolerance).
00019  *
00020  * SPDX-License-Identifier: MIT
00021  * @copyright Copyright (c) 2026 Locked Inc.
00022  */
00023
00024 #include <stdint.h>
00025
00026 #include "rx72n_regs.h"
00027 #include "rx_register_protection.h"
00028
00029 /*
      ================================================================================
00030  * SCI9 register access via raw addresses (rx72n_sci_regs.h doesn't expose
00031  * the per-channel struct as cleanly as we'd like for a quick polled driver).
00032  * The base 0x000D0020 is taken from the verified header.
00033  * Register offsets per RX72N HW manual: SMR=0, BRR=1, SCR=2, TDR=3, SSR=4.
00034  *
      ================================================================================
      */
00035 typedef enum : uintptr_t {
00036     k_sci9_base = 0x000D0020U,
00037 } sci9_addrs_t;
00038
00039 typedef enum : uint8_t {
```

```
00040     k_psel_sci9        = 0x0AU, /**< SCI9 RXD/TXD function */
00041     k_pb6_mask         = (uint8_t)(1U << 6U),
00042     k_pb7_mask         = (uint8_t)(1U << 7U),
00043     k_mstpc_sci9_bit   = 26U, /* MSTPCRC.MSTPC26 per RX72N HW manual page 410 */
00044     k_pwpr_unlock_b0   = 0x00U,
00045     k_pwpr_pfswe       = 0x40U,
00046     k_pwpr_b0wi        = 0x80U,
00047     k_brr_115200       = 31U, /* SCI9 uses PCLKA=120 MHz (SCI7-11 group); BRR=31 @ CKS=0 ABCS=0 -> ~117
      kbaud (1.7% over 115200) */
00048     k_smr_8n1_async    = 0x00U,
00049     k_scr_te_only      = 0x20U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=0 */
00050     k_scr_txrx_enabled = 0x30U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=1 */
00051     k_scr_disabled     = 0x00U,
00052     k_ssr_tdre         = 0x80U,
00053     k_scmr_default     = 0xF2U,
00054     k_semr_default     = 0x00U,
00055     k_hex_nibble_mask  = 0x0FU,
00056 } sci9_cfg_t;
00057
00058 typedef enum : uint16_t {
00059     k_brr_settle_loops = 1000U, /**< Match production uart.c (~8.68 us @ 115200) */
00060 } sci9_settle_t;
00061
00062 typedef enum : uint8_t {
00063     k_hex_shift_init32 = 28U,
00064     k_hex_shift_init16 = 12U,
00065     k_hex_shift_step   = 4U,
00066 } hex_shifts_t;
00067
00068 /*
      ============================================================================
00069  * SCI9 register accessors (raw addresses since the header doesn't give us a
00070  * per-channel struct for the SCIi/SCIj quirks).
00071  *
      ============================================================================
      */
00072 static inline volatile uint8_t *sci9_smr(void)
00073 {
00074     return (volatile uint8_t *)(k_sci9_base + 0U);
00075 }
00076 static inline volatile uint8_t *sci9_brr(void)
00077 {
00078     return (volatile uint8_t *)(k_sci9_base + 1U);
00079 }
00080 static inline volatile uint8_t *sci9_scr(void)
00081 {
00082     return (volatile uint8_t *)(k_sci9_base + 2U);
00083 }
00084 static inline volatile uint8_t *sci9_tdr(void)
00085 {
00086     return (volatile uint8_t *)(k_sci9_base + 3U);
00087 }
00088 static inline volatile uint8_t *sci9_ssr(void)
00089 {
00090     return (volatile uint8_t *)(k_sci9_base + 4U);
00091 }
00092 static inline volatile uint8_t *sci9_scmr(void)
00093 {
00094     return (volatile uint8_t *)(k_sci9_base + 6U);
00095 }
00096 static inline volatile uint8_t *sci9_semr(void)
00097 {
00098     return (volatile uint8_t *)(k_sci9_base + 7U);
00099 }
00100
00101 /*
      ============================================================================
00102  * Initialise SCI9 for 115200 8N1 polled output on PB7 (TXD9).
00103  *
      ============================================================================
      */
00104 void sci9_debug_init(void)
00105 {
00106     /* Step 1: release SCI9 from module stop. SCI9 lives at MSTPCRC bit 26
00107      * (NOT MSTPCRB.bit22 -- that's PDC). PRC1 unlock gates MSTPCR* writes. */
00108     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109     system_regs()->mstpcrc &= ~(uint32_t)(1UL << (uint32_t)k_mstpc_sci9_bit);
00110     *prcr_reg() = k_rx_prcr_lock;
00111
00112     /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC.  Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear B0WI */
00116     mpc()->pwpr   = k_pwpr_pfswe;     /* allow PFS writes */
00117     mpc()->pb6pfs = k_psel_sci9;
00118     mpc()->pb7pfs = k_psel_sci9;
00119     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear PFSWE */
```

```
00120     mpc()->pwpr   = k_pwpr_b0wi;      /* re-protect */
00121
00122     /* Step 3: PDR before PMR (production uart.c does this).  PB7 = output
00123      * for TX, PB6 = input for RX.  Then flip PMR to peripheral mode. */
00124     portb()->pdr |= (uint8_t)k_pb7_mask;
00125     portb()->pdr &= (uint8_t) ~k_pb6_mask;
00126     portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);
00127
00128     /* Step 4: SCI9 init.  Match the production order:
00129      *   SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130      * Don't touch SCMR (reset default 0xF2 is correct). */
00131     *sci9_scr() = k_scr_disabled;
00132     *sci9_smr() = k_smr_8n1_async;
00133     *sci9_brr() = k_brr_115200;
00134
00135     /* Wait roughly one bit time before enabling TX/RX. */
00136     for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
00137         __asm__ volatile("nop");
00138     }
00139
00140     *sci9_scr()  = k_scr_txrx_enabled;
00141     *sci9_semr() = k_semr_default;
00142 }
00143
00144 /*
    ============================================================================
00145  * Polled write: spin until TDRE then drop byte into TDR.
00146  *
    ============================================================================
    */
00147 void sci9_debug_putc(char c)
00148 {
00149     while ((*sci9_ssr() & (uint8_t)k_ssr_tdre) == 0U) {
00150         /* spin */
00151     }
00152     *sci9_tdr() = (uint8_t)c;
00153 }
00154
00155 void sci9_debug_puts(const char *s)
00156 {
00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s++);
00165     }
00166 }
00167
00168 void sci9_debug_puthex32(uint32_t v)
00169 {
00170     static const char hex_digits[] = "0123456789ABCDEF";
00171     sci9_debug_putc('0');
00172     sci9_debug_putc('x');
00173     for (int8_t shift = (int8_t)k_hex_shift_init32; shift >= 0;
00174          shift -= (int8_t)k_hex_shift_step) {
00175         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00176     }
00177 }
00178
00179 void sci9_debug_puthex16(uint16_t v)
00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
00185          shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }
```